

Theoretical CompSci Cheat Sheet

Formal Language Fundamentals

Core Definitions

Concept	Formal Definition	Notes
Alphabet (Σ)	A finite, non-empty set of symbols.	Binary $\Sigma = \{0, 1\}$.
Word (w)	A finite sequence of letters from Σ .	Length $ w $ is the symbol count; $ \epsilon = 0$.
Concatenation ($u \cdot w$)	Laying letters of w after u : $u_1 \dots u_n w_1 \dots w_m$.	Neutral element: $w\epsilon = \epsilon w = w$.
Kleene Star (Σ^*)	$\Sigma^* = \bigcup_{n=0}^{\infty} \Sigma^n = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \dots$	Set of all finite words over Σ , including ϵ .
Language (L)	Any subset $L \subseteq \Sigma^*$.	Closed under standard set operations ($\cup, \cap, \setminus, \overline{L}$).

Automata Formulations

Component	DFA $A=(\Sigma, Q, \delta, q_0, F)$	NFA $A=(\Sigma, Q, \Delta, I, F)$
States (Q)	Finite set of states	Finite set of states
Start State	Exactly one: $q_0 \in Q$	Set of initial states: $I \subseteq Q$
Accepting States (F)	Subset $F \subseteq Q$	Subset $F \subseteq Q$
Transition Function	$\delta : Q \times \Sigma \rightarrow Q$ (deterministic)	$\Delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ (multi-path or dead ends)
Extended Transition	$\delta^*(\epsilon, q) = q$ $\delta^*(wa, q) = \delta(\delta^*(w, q), a)$	$\Delta^*(\epsilon, q) = \{q\}$ $\Delta^*(wa, q) = \bigcup_{p \in \Delta^*(w, q)} \Delta(p, a)$
Accepted Language	$L(A) = \{w \in \Sigma^* \mid \delta^*(w, q_0) \in F\}$	$L(A) = \{w \in \Sigma^* \mid (\bigcup_{i \in I} \Delta^*(w, i)) \cap F \neq \emptyset\}$

Inductive Proof Template: DFA Correctness

Use this template to prove an automaton A computes an intended invariant $P(w)$ (such as parity or remainder modulo k).

- **Claim:** Let $P : \Sigma^* \rightarrow Q$ denote the state assigned to the invariant property of word w . For all $n \in \mathbb{N}$ and all words $w \in \Sigma^*$ with $|w| = n$, $\delta^*(w, q_0) = P(w)$.
- **Base Case ($n = 0$):**
 - The only word of length 0 is $w = \epsilon$.
 - By the extended transition definition, $\delta^*(\epsilon, q_0) = q_0$.
 - Verify that $q_0 = P(\epsilon)$ holds by problem definition.
- **Inductive Hypothesis:**
 - Assume for some $n \geq 0$, $\delta^*(u, q_0) = P(u)$ holds for every word $u \in \Sigma^*$ with $|u| = n$.
- **Inductive Step:**
 - Consider a word $w \in \Sigma^*$ of length $n + 1$. Decompose w as $w = ua$ where $|u| = n$ and $a \in \Sigma$.
 - Expand using the inductive definition:
$$\delta^*(w, q_0) = \delta^*(ua, q_0) = \delta(\delta^*(u, q_0), a)$$
 - Apply the Induction Hypothesis to substitute $\delta^*(u, q_0)$:
$$\delta(\delta^*(u, q_0), a) = \delta(P(u), a)$$
 - Evaluate the machine's transition $\delta(P(u), a)$ and show it equals $P(ua) = P(w)$.
- **Conclusion:** By mathematical induction, the claim holds for all $n \geq 0$. Since $w \in L(A) \iff \delta^*(w, q_0) \in F \iff P(w) \in F, L(A) = L$.

Algorithms

Determinization: NFA to DFA (Subset Construction)

Converts an NFA $A = (\Sigma, Q, \Delta, I, F)$ into an equivalent DFA $A^D = (\Sigma, Q^D, \delta^D, q_0^D, F^D)$.

```

Algorithm Subset_Construction:
  Input: NFA A = (Sigma, Q, Delta, I, F)
  Output: DFA A^D = (Sigma, Q^D, delta^D, q_0^D, F^D)

  q_0^D := I
  Q^D := { q_0^D }
  WorkList := [ q_0^D ]

  While WorkList is not empty:
    Pop S from WorkList
    For each symbol a in Sigma:
      TargetSet := Union_{q in S} Delta(q, a)
      delta^D(S, a) := TargetSet
      If TargetSet not in Q^D:
        Add TargetSet to Q^D
        Push TargetSet to WorkList

  F^D := { S in Q^D | S \cap F != empty_set }

```

DFA Minimization (Table-Filling / Hopcroft-Moore Variant)

Computes the minimal canonical DFA $A^{\min}$ by collapsing indistinguishable states ($L_p = L_q$).

- Step 1: Trim.** Eliminate unreachable states (run BFS/DFS from q_0) and dead states (states that cannot reach any $f \in F$).
- Step 2: Initialize Table.** Construct an upper-triangular table of all state pairs $\{p, q\}$ where $p \neq q$.
- Step 3: Base Mark ($k = 0$).** Mark any pair $\{p, q\}$ where one state is accepting and the other is non-accepting ($p \in F \oplus q \in F$). These are distinguished by ϵ .
- Step 4: Inductive Mark Propagation.**
 - For each unmarked pair $\{p, q\}$ and each symbol $a \in \Sigma$:
 - Find target states $p' = \delta(p, a)$ and $q' = \delta(q, a)$.
 - If $p' \neq q'$ and the pair $\{p', q'\}$ is already marked, mark $\{p, q\}$ with the next step index $k + 1$.
 - Repeat until a full pass over all unmarked pairs yields no new markings.
- Step 5: Partition & Construct Quotient Machine.**
 - Any pair $\{p, q\}$ left unmarked is indistinguishable ($p \sim q$).
 - Form equivalence classes $[q]$ from unmarked relations.
 - New DFA: $q_0^{\min} = [q_0]$, $F^{\min} = \{[q] \mid q \in F\}$, and $\delta^{\min}([q], a) = [\delta(q, a)]$.

Checking Equivalence of Two Languages: $L(A) \stackrel{?}{=} L(B)$

Plaintext

```

Algorithm Check_Language_Equivalence:
  Input: Automata A and B (NFAs or DFAs)
  1. If either is an NFA, convert to DFA via Subset Construction.
  2. Minimize DFA_A to obtain canonical A_min.
  3. Minimize DFA_B to obtain canonical B_min.
  4. Compare A_min and B_min:
    - Check graph isomorphism up to state renaming (match start states, transitions, accepting states).
    - If isomorphic: L(A) = L(B).
    - Otherwise: L(A) != L(B).

```

Regular Expressions Reference

A language is **regular** if and only if it is recognized by a DFA, an NFA, or matched by a Regular Expression.

Inductive Definition of REs over Σ

- Base Cases:** $\emptyset$ (empty language), ϵ (empty string $\{\epsilon\}$), and a for each $a \in \Sigma$ ($\{a\}$).
- Inductive Steps:** If r_1, r_2 are REs:
 - Union: $(r_1 + r_2)$ or $(r_1 \mid r_2)$
 - Concatenation: $(r_1 \cdot r_2)$ or simply $r_1 r_2$

- Kleene Star: r_1^* (zero or more repetitions)
- **Precedence:** Star (*) > Concatenation (.) > Union (+)

Structural Translation: RE to NFA (Thompson's Construction)

Plaintext

Base Cases:

```
r = empty_set: (start)          (accept)          [no transitions]
r = epsilon:   (start) --- epsilon ----> ((accept))
r = a:         (start) ----- a -----> ((accept))
```

Inductive Constructions:

```
r = r1 + r2:   Split new start state via epsilon to Start(r1) and Start(r2).
               Connect all Accept(r1) and Accept(r2) via epsilon to new single accept state.

r = r1 * r2:   Link Accept(r1) via epsilon transitions directly to Start(r2).
               Start is Start(r1); accept states are Accept(r2).

r = (r1)*:     Add new start q0 and new accept qf.
               Add epsilon transitions: q0 -> Start(r1), q0 -> qf (zero matches),
               Accept(r1) -> Start(r1) (loop back), Accept(r1) -> qf.
```

Midterm 1 Synthesis: Comprehensive Example

Problem: Construct an automaton over $\Sigma = \{0, 1\}$ for $L = \{w \mid w \text{ has an even number of 1s or contains } 01 \text{ as a factor}\}$.

- **Decomposition via Sub-languages:**
 - Let $L_1 = \{w \mid \text{count}_1(w) \equiv 0 \pmod{2}\}$.
 - Let $L_2 = \{w \mid w = x01y \text{ for } x, y \in \{0, 1\}^*\} = \Sigma^*01\Sigma^*$.
 - Overall Language: $L = L_1 \cup L_2$.
- **Step-by-Step Execution Plan:**
 1. **NFA Construction:** Create an NFA with initial state branching via ϵ -transitions to the start of A_{L_1} (a 2-state parity cycle) and the start of A_{L_2} (a 3-state pattern matcher for 01).
 2. **Determinization:** Apply the Subset Construction table. Start at $S_0 = I$, expand transitions on 0 and 1 , and label states containing any original final states as accepting.
 3. **Minimization:** Fill the upper-triangular table to identify and collapse redundant states (e.g., once the factor 01 is detected and parity is satisfied).

asdf